Top 50 Linux Interview Questions

for DevOps Engineers — Structured Answers

A point-by-point reference guide covering Linux fundamentals, file systems, permissions, process management, memory, storage (LVM), networking, and real-world production troubleshooting scenarios commonly asked in DevOps / SRE interviews.

1. Linux Basics & File System

Q1. What is Linux and why is it used in DevOps?

- Linux is a free, open-source, Unix-like operating system kernel, with distributions (Ubuntu, CentOS, RHEL, Debian) built around it.
- It is highly stable, secure, customizable, and lightweight compared to proprietary operating systems.
- Most cloud platforms, containers (Docker), and orchestration tools (Kubernetes) run natively on Linux.
- DevOps tooling (Ansible, Jenkins, Terraform, shell scripting) is built around Linux command-line operations.
- Open-source nature allows automation, scripting, and fine-grained control needed for CI/CD pipelines.

Q2. Explain the Linux file system hierarchy.

- / : Root directory; the top of the entire file system tree.
- /bin, /sbin : Essential user and system binaries/commands.
- /etc : System-wide configuration files.
- /home : Personal directories for each user.
- /var : Variable data such as logs, spool files, and caches.
- /tmp : Temporary files, cleared periodically.
- /usr : User-installed applications and libraries.
- /opt : Optional/third-party software packages.
- /proc, /sys : Virtual file systems exposing kernel and process information.
- /dev : Device files representing hardware.

Q3. What is the difference between Hard Link and Soft Link?

- Hard Link: Points directly to the same inode as the original file; shares the same data blocks.
- Hard Link: Cannot link across different file systems/partitions, and cannot link to directories.
- Soft Link (Symbolic Link): A separate file containing a path/pointer to the target file.
- Soft Link: Can span across file systems and can link to directories.
- If the original file is deleted, a hard link still works; a soft link becomes broken (dangling).

Q4. What are inode numbers?

- An inode is a data structure that stores metadata about a file (permissions, owner, size, timestamps, and pointers to data blocks).
- It does NOT store the file name; the name is mapped to the inode number in the directory entry.
- Each file system has a finite number of inodes; running out of inodes prevents new file creation even if disk space remains.
- Command to view: ls -i or df -i (for inode usage per filesystem).

Q5. What is the difference between cp, mv, and rsync?

- cp : Copies files/directories; creates a duplicate, original remains unchanged.
- mv : Moves or renames files/directories; original location no longer holds the file.
- rsync : Synchronizes files/directories, copying only the differences (delta transfer).
- rsync supports remote sync over SSH, resuming interrupted transfers, and preserving permissions/timestamps — ideal for large datasets and backups.
- cp and mv perform full copies each time; rsync is far more efficient for repeated syncs.

Q6. What is the difference between df and du?

- df (disk free) : Reports filesystem-level disk space usage (total, used, available, mount points).
- du (disk usage) : Reports the disk space consumed by specific files/directories.

- df -h shows human-readable filesystem space; du -sh shows the size of a directory.
- Discrepancies between df and du often indicate open file handles for deleted files still holding space.

Q7. How do you find large files on a Linux server?

- du -ah /path | sort -rh | head -20 — lists largest files/directories under a path.
- find / -type f -size +100M — finds files larger than 100MB.
- ncdu — interactive disk usage analyzer (if installed).
- lsof | grep deleted — finds large deleted-but-open files still consuming space.

Q8. What are the purposes of /etc, /var, /tmp, and /proc directories?

- /etc : Stores configuration files for the system and installed applications (e.g., /etc/passwd, /etc/hosts).
- /var : Stores variable/runtime data like logs (/var/log), mail spools, and cache.
- /tmp : Temporary storage for files needed only during a session; typically cleared on reboot.
- /proc : A virtual filesystem providing real-time information about running processes and kernel parameters (e.g., /proc/cpuinfo, /proc/meminfo).

2. Users, Permissions & Security

Q9. How do you create, modify, and delete users?

- Create: useradd -m -s /bin/bash username (then set password with passwd username).
- Modify: usermod -aG groupname username (add to group), usermod -l newname oldname (rename).
- Delete: userdel -r username (the -r flag also removes the home directory).
- View user details: id username or grep username /etc/passwd.

Q10. Explain Linux file permissions (rwx).

- r (read) : View file contents or list directory contents.
- w (write) : Modify/delete file contents or add/remove files in a directory.
- x (execute) : Run a file as a program/script, or enter (cd into) a directory.
- Permissions are grouped into three sets: Owner, Group, and Others, e.g., rwxr-xr--.
- Viewed using ls -l; changed using chmod.

Q11. What is the difference between chmod and chown?

- chmod : Changes the permission bits (read/write/execute) of a file or directory.
- chown : Changes the ownership (user and/or group) of a file or directory.
- Example: chmod 755 file.sh sets rwxr-xr-x permissions.
- Example: chown user:group file.sh changes both owner and group.

Q12. What are SUID, SGID, and Sticky Bit?

- SUID (Set User ID) : When set on an executable, it runs with the privileges of the file owner, not the executing user (e.g., /usr/bin/passwd).
- SGID (Set Group ID) : On executables, runs with the group's privileges; on directories, new files inherit the directory's group.
- Sticky Bit : Set on directories (e.g., /tmp) so only the file owner (or root) can delete/rename files within it, even if others have write access.
- Set using chmod u+s, chmod g+s, and chmod +t respectively, or numeric prefixes 4, 2, 1.

Q13. What is umask?

- umask defines the default permission mask subtracted from maximum permissions when new files/directories are created.
- Default max permissions: 666 for files, 777 for directories.
- A umask of 022 results in default file permissions of 644 and directory permissions of 755.
- View current value with umask; set with umask 027 (commonly configured in /etc/profile or ~/.bashrc).

Q14. What is the difference between /etc/passwd and /etc/shadow?

- /etc/passwd : Stores user account information — username, UID, GID, home directory, and default shell. World-readable.
- /etc/shadow : Stores encrypted password hashes and password aging information. Readable only by root for security.
- Separating the two prevents ordinary users from accessing password hashes, reducing risk of offline cracking.

Q15. How do you grant sudo access to a user?

- Add the user to the sudo (Debian/Ubuntu) or wheel (RHEL/CentOS) group: usermod -aG sudo username.
- Alternatively, edit /etc/sudoers (preferably via visudo to avoid syntax errors) and add a specific rule, e.g., username ALL=(ALL) ALL.
- For granular access, restrict to specific commands: username ALL=(ALL) /usr/bin/systemctl restart nginx.
- Always use visudo rather than directly editing the file, as it validates syntax before saving.

3. Process Management

Q16. What is a process in Linux?

- A process is an instance of a running program, with its own memory space, PID (Process ID), and execution context.
- Each process has a parent process (PPID) except the root process (init/systemd, PID 1).
- Processes can be in states such as Running, Sleeping, Stopped, or Zombie.
- Viewed using ps, top, or htop.

Q17. What is the difference between a process and a thread?

- Process : An independent execution unit with its own memory space; isolated from other processes.
- Thread : A lightweight unit of execution within a process; threads of the same process share memory and resources.
- Context switching between threads is faster and cheaper than between processes.
- Communication between processes (IPC) is more complex than communication between threads (shared memory).

Q18. Explain ps, top, and htop commands.

- ps : Displays a snapshot of currently running processes (e.g., ps -ef, ps aux).
- top : Provides a real-time, continuously updating view of system processes, CPU, and memory usage.
- htop : An enhanced, interactive, color-coded version of top with mouse support and easier process management (kill, sort, filter).
- ps is best for scripting; top/htop are best for live interactive monitoring.

Q19. What is the difference between kill, killall, and pkill?

- kill : Sends a signal to a process using its specific PID (e.g., kill -9 1234).
- killall : Sends a signal to all processes matching an exact process name (e.g., killall nginx).

- pkill : Sends a signal to processes matching a pattern (name, user, or other attributes), supporting more flexible matching (e.g., pkill -u username).
- Signal 9 (SIGKILL) forces termination; signal 15 (SIGTERM, default) requests graceful termination.

Q20. How do you identify a high CPU-consuming process?

- Run top or htop and sort by CPU usage (Shift+P in top).
- Use ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head to list top CPU consumers via command line.
- Use pidstat 1 for per-process CPU statistics over time.
- Cross-check with application logs to correlate spikes with specific requests or jobs.

Q21. What is process priority?

- Priority determines the order/weight in which the kernel scheduler allocates CPU time to processes.
- Linux uses priority values ranging typically from -20 (highest priority) to 19 (lowest priority).
- Higher priority processes get more CPU time relative to others when resources are contended.
- Viewed in the PR column of top.

Q22. What are nice and renice commands?

- nice : Starts a new process with a specified priority (niceness) value, e.g., nice -n 10 ./script.sh.
- renice : Changes the priority of an already running process, e.g., renice -n 5 -p 1234.
- Lower niceness values (down to -20) increase priority; higher values (up to 19) decrease it.
- Only root can set negative (higher priority) niceness values.

Q23. What are Zombie and Orphan processes?

- Zombie process : A process that has completed execution but still has an entry in the process table because its parent hasn't read its exit status (state Z).
- Zombies consume no CPU/memory resources but occupy a PID slot; they are cleared once the parent calls wait().
- Orphan process : A process whose parent has terminated before it; it gets re-parented to init/systemd (PID 1), which then handles its cleanup.
- Excessive zombies usually indicate a bug in the parent application's process-handling logic.

4. Memory & Storage

Q24. How do you check memory utilization?

- free -h : Shows total, used, free, shared, buffer/cache, and available memory in human-readable form.
- top / htop : Shows live memory usage alongside CPU usage.
- cat /proc/meminfo : Detailed, granular memory statistics directly from the kernel.
- vmstat 1 : Shows memory, swap, and I/O statistics at intervals.

Q25. What is swap memory?

- Swap is disk space used as an extension of RAM when physical memory is exhausted.
- The kernel moves inactive memory pages to swap space, freeing RAM for active processes.
- Excessive swapping ('thrashing') significantly degrades performance since disk I/O is far slower than RAM.
- Checked using free -h, swapon -s, or top.

Q26. What is OOMKilled?

- OOM (Out Of Memory) Killer is a kernel mechanism that terminates processes when the system runs critically low on memory.

- It selects a process to kill based on an 'oom_score', typically targeting the largest memory consumer.
- In containerized environments (e.g., Kubernetes), a pod exceeding its memory limit shows status 'OOMKilled'.
- Logs can be checked using dmesg | grep -i oom or /var/log/messages.

Q27. How do you identify memory leaks?

- Monitor memory usage trends over time using top, htop, or monitoring tools (Prometheus/Grafana) to spot continuous growth without release.
- Use tools like valgrind, pmap , or smem for in-depth process memory analysis.
- Check for gradually increasing RSS (Resident Set Size) for a specific process via ps -o pid,vsz,rss,cmd.
- Correlate with application-level profiling/logs to pinpoint the leaking component.

Q28. What is LVM and why is it used?

- LVM (Logical Volume Manager) abstracts physical storage into Physical Volumes (PV), Volume Groups (VG), and Logical Volumes (LV).
- Allows dynamic resizing (extending/shrinking) of storage without unmounting or repartitioning.
- Enables flexible storage management: striping, snapshots, and combining multiple disks into a single volume.
- Commonly used in production servers for scalable and flexible disk management.

Q29. What is the difference between ext4 and xfs?

- ext4 : Widely-used, mature journaling file system; good general-purpose performance and broad tool support.
- xfs : High-performance journaling file system optimized for large files and parallel I/O; default on many RHEL/CentOS systems.
- xfs supports online resizing (grow only, not shrink); ext4 supports both grow and shrink (offline).
- xfs generally performs better with very large files and high-throughput workloads; ext4 is often preferred for general-purpose and smaller systems.

Q30. How do you mount and unmount a filesystem?

- Mount: mount /dev/sdb1 /mnt/data — attaches a filesystem to a directory (mount point).
- Unmount: umount /mnt/data — detaches the filesystem (use umount -l for lazy unmount if busy).
- Persistent mounts are configured in /etc/fstab so they survive reboots.
- Check active mounts using mount, df -h, or findmnt.

Q31. How do you check inode utilization?

- df -i : Shows inode usage (used/available/percentage) per mounted filesystem.
- 100% inode utilization prevents creating new files even if disk space (df -h) shows free space.
- Common cause: a directory with millions of tiny files (e.g., session/cache files, logs).
- Resolved by deleting unnecessary small files or reformatting with more inodes allocated.

Q32. How do you troubleshoot disk space issues?

- Run df -h to identify which filesystem/partition is full.
- Use du -sh /* (or per directory) to locate the largest consumers of space.
- Check for large or rotated log files in /var/log and old core dumps.
- Check for deleted-but-open files using lsof | grep deleted that still hold disk space.
- Clear package caches, temp files, and old kernel versions if applicable, then verify with df -h again.

5. Networking

Q33. How do you check IP addresses in Linux?

- ip addr show (or ip a) — modern command showing all interfaces and their IPs.
- ifconfig — older/legacy command (may need net-tools package installed).
- hostname -I — quickly lists assigned IP addresses.
- nmcli device show — shows IP details via NetworkManager.

Q34. What is the difference between netstat and ss?

- netstat : Legacy tool for displaying network connections, routing tables, and interface statistics.
- ss (socket statistics) : Modern, faster replacement for netstat, pulling data directly from kernel space.
- ss is generally preferred in modern distributions since netstat is deprecated/often unavailable by default.
- Example: ss -tulnp lists listening TCP/UDP ports with process info, equivalent to netstat -tulnp.

Q35. How do you check whether a port is listening?

- ss -tulnp | grep — shows listening TCP/UDP ports and owning processes.
- netstat -tulnp | grep — legacy equivalent.
- lsof -i : — shows the process bound to a specific port.
- telnet or nc -zv — tests connectivity to a port from a remote/local client.

Q36. What is the difference between ping, traceroute, and nslookup?

- ping : Tests basic reachability and latency to a host using ICMP echo requests.
- traceroute : Maps the network path (hop-by-hop) a packet takes to reach a destination, helping identify where delays/failures occur.
- nslookup : Queries DNS servers to resolve domain names to IP addresses (or vice versa).
- Used together: ping confirms reachability, traceroute isolates the failing hop, nslookup verifies DNS resolution.

Q37. How do you troubleshoot DNS issues?

- Verify resolution using nslookup or dig .
- Check /etc/resolv.conf for correct nameserver entries.
- Confirm /etc/hosts doesn't contain conflicting static entries.
- Test against a public DNS (e.g., dig @8.8.8.8 domain.com) to isolate local resolver issues vs. external DNS issues.
- Check DNS server logs and network connectivity to the DNS server (port 53).

Q38. How do you check established TCP connections?

- ss -tan state established — lists all established TCP connections.
- netstat -tan | grep ESTABLISHED — legacy equivalent.
- lsof -i tcp -s tcp:established — lists established connections with owning processes.
- Useful for diagnosing connection leaks or excessive open connections to a database/service.

Q39. What is the difference between SCP and RSYNC?

- SCP (Secure Copy) : Copies files securely over SSH; performs a full copy every time, no resume capability.
- RSYNC : Transfers only the differences (deltas) between source and destination, supports resuming interrupted transfers.
- RSYNC is significantly more efficient for repeated transfers or large datasets; SCP is simpler for one-off transfers.
- RSYNC over SSH: rsync -avz -e ssh /local/path user@remote:/remote/path.

Q40. How do you capture packets using tcpdump?

- Basic capture: tcpdump -i eth0 — captures live traffic on a specific interface.
- Filter by port: tcpdump -i eth0 port 443 — captures HTTPS traffic only.

- Save to file for analysis: tcpdump -i eth0 -w capture.pcap (analyze later with Wireshark).
- Filter by host: tcpdump -i eth0 host 10.0.0.5 — captures traffic to/from a specific IP.

Q41. How do you troubleshoot SSH connection failures?

- Verify the SSH service is running on the target: systemctl status sshd.
- Check connectivity/port reachability: telnet 22 or nc -zv 22.
- Review /var/log/auth.log or /var/log/secure for authentication failures.
- Verify SSH key permissions (chmod 600 for private keys) and correct username/IP.
- Use verbose mode for debugging: ssh -vvv user@host to see the detailed handshake process.

6. Production Troubleshooting Scenarios

Q42. Server CPU is 100%. How will you troubleshoot?

- Run top/htop to identify the process(es) consuming the most CPU.
- Check if the spike is from a single runaway process or distributed across many (e.g., due to traffic surge).
- Use pidstat or strace -p to inspect what the process is doing.
- Check for recent deployments, cron jobs, or scheduled batch jobs that may have triggered the spike.
- Mitigate by renice-ing or restarting the offending process, then perform root-cause analysis from logs/application metrics.

Q43. Disk utilization is 100%. What steps will you follow?

- Run df -h to confirm which mount point/partition is full.
- Use du -sh /* to drill down and find the largest space consumers.
- Check /var/log for oversized or unrotated log files; rotate/compress/delete as needed.
- Check for deleted-but-still-open file handles using lsof | grep deleted.
- Clear temp files, old backups, and unused container images/layers; expand the volume via LVM if a long-term fix is needed.

Q44. Application is down, but the process is running. How will you investigate?

- Check application logs for errors, exceptions, or hung threads.
- Verify the application is actually listening on its expected port: ss -tulnp | grep .
- Test connectivity locally (curl localhost:) vs. externally to isolate network vs. application issues.
- Check for resource exhaustion (CPU, memory, open file descriptors, DB connections) causing the app to be unresponsive.
- Check dependent services (database, cache, message queue) for outages affecting the application's health checks.

Q45. Port 8080 is inaccessible. How will you troubleshoot?

- Confirm the service is actually listening: ss -tulnp | grep 8080.
- Test locally first: curl localhost:8080 to rule out application-level failure.
- Check firewall rules: iptables -L, firewalld, or cloud security groups/NACLs blocking the port.
- Verify no other process is already bound to the port causing a conflict.
- Use telnet/nc from a remote host to determine if it's a network/firewall block vs. an application issue.

Q46. How do you analyze logs in Linux?

- Use tail -f /var/log/app.log to monitor logs in real-time.
- Use grep, awk, and sed to filter and extract relevant patterns/errors from large log files.

- Use journalctl -u for systemd-managed service logs.
- Use centralized logging tools (ELK/EFK stack, Grafana Loki, Splunk) for aggregated, searchable log analysis across multiple servers.
- Correlate timestamps across application, system, and access logs to build a timeline of the incident.

Q47. How do you identify a memory issue in production?

- Check overall memory and swap usage with free -h and watch for high swap utilization.
- Identify top memory-consuming processes using top/htop sorted by memory (Shift+M).
- Check for OOM kills in dmesg | grep -i oom or /var/log/messages.
- Monitor memory trends over time using Prometheus/Grafana to detect gradual leaks vs. sudden spikes.
- Inspect application-level metrics/heap dumps if the issue is within a specific service.

Q48. A filesystem becomes read-only. How will you recover it?

- Check dmesg or /var/log/messages for filesystem errors (often caused by disk corruption or I/O errors).
- Identify the affected mount: mount | grep ro, and confirm read-only state.
- Remount in read-write mode (if disk integrity allows): mount -o remount,rw /mountpoint.
- Run a filesystem check on the affected partition (after unmounting, where possible): fsck /dev/sdXN.
- If hardware (disk) failure is suspected, plan for data recovery/migration and replace the failing disk.

Q49. A server becomes slow after deployment. How will you find the root cause?

- Compare resource usage (CPU, memory, disk I/O, network) before and after the deployment using monitoring dashboards.
- Review the deployment diff/changelog for configuration changes, new dependencies, or inefficient code.
- Check application and system logs around the deployment timestamp for new errors or warnings.
- Use strace, iostat, and vmstat to identify whether the bottleneck is CPU, memory, disk I/O, or network-bound.
- If confirmed deployment-related, consider rolling back while root cause analysis continues.

Q50. How do you perform Linux troubleshooting in production environments?

- Follow a structured approach: Identify symptoms -> Gather data (logs/metrics) -> Form hypothesis -> Test -> Resolve -> Document (post-mortem).
- Use monitoring/observability tools (Prometheus, Grafana, ELK) to get real-time visibility before manually SSHing into servers.
- Prioritize non-destructive diagnostic commands first (top, df, free, journalctl) before making changes.
- Always check recent changes first: deployments, configuration changes, scaling events, or scheduled jobs.
- Communicate status during major incidents and document the root cause and remediation steps afterward to prevent recurrence.

Note: For 1-to-1 mentorship on DevOps/SRE/Cloud career transitions and interview preparation, reach out directly for personalized guidance.